1. Hibernate Validator `ConstraintHelper` Parsing Error

Error message:

“Error encountered while parsing org.hibernate.validator.internal.metadata.core.ConstraintHelper.resolve”

Why it happens: Hibernate Validator uses reflection and class initialization patterns that GraalVM cannot fully resolve at build time .

The fix you discovered:

```
bash
./mvnw package -Pnative -DskipTests
-Dquarkus.native.additional-build-args=--initialize-at-run-time=org.hibernate.validator.internal.constraintvalidators.bv.time
```

---

2. Missing Reflection Registration for Serialization

Error message:

“t
Cannot construct instance of `org.acme.MyClassName` (no Creators, like default constructor, exist)”

Why it happens: When your code uses libraries that rely on reflection for serialization (Jackson, Avro, etc.), GraalVM cannot see those classes at build time .

The fix:

```
java
import io.quarkus.runtime.annotations.RegisterForReflection;

@RegisterForReflection(targets = {MyClass.class})
public class ReflectionConfiguration {
    // Empty class - just holds the annotation
}
```

---

3. `/etc/hosts` DNS Resolution Delay (macOS/Linux)

Symptom: Native executable starts but takes 5+ seconds before any output appears. The `BannerProcessor.recordBuildStep` shows 5+ seconds in logs.

Why it happens: Your machine's hostname is not properly mapped to `127.0.0.1` in `/etc/hosts`. The application attempts DNS resolution that times out after several seconds.

The fix:

```
bash
# Find your hostname
hostname

# Ensure /etc/hosts contains:
127.0.0.1       localhost your-hostname
::1             localhost your-hostname
```

---

4. Random/SplittableRandom Seed Error

Error message:

“
Detected an instance of Random/SplittableRandom class in the image heap. Instances created during image generation have cached seed values and don't behave as expected.”

Why it happens: Some libraries create `Random` or `SecureRandom` instances during static initialization. These get frozen into the native image at build time, causing unpredictable behavior at runtime .

The fix:

```
bash
./mvnw package -Pnative -DskipTests
-Dquarkus.native.additional-build-args=--initialize-at-run-time=java.security.SecureRandom
```

---

5. Dynamic Proxy Reflection Error

Error message:

“The program tried to reflectively access the proxy class inheriting [<>] without it being registered for runtime reflection.”

Why it happens: Libraries that create dynamic proxies at runtime (like CXF, some HTTP clients, or AWT) need those proxy interfaces registered for reflection in the native image .

The fix: Register the dynamic proxy interfaces:

java
```
@BuildStep
void registerProxies(BuildProducer<NativeImageProxyDefinitionBuildItem> proxy) {
    proxy.produce(new NativeImageProxyDefinitionBuildItem(MyInterface.class.getName()));
}
```

Or add to `application.properties`:

properties
```
quarkus.native.additional-build-args=--initialize-at-run-time=org.apache.cxf.*
```

```
@RegisterForReflection(targets = {MyClass.class})
public class ReflectionConfiguration {
    // Empty class - just holds the annotation
}
```
